# PSM: Prompt Sensitivity Minimization via LLM-Guided Black-Box Optimization

**Hussein Jawad**[1], **Nicolas J-B. Brunel**[1,2,3]

[1]Capgemini Invent, Paris, France
[2]LaMME, University Paris Saclay, Evry, France
[3]ENSIIE, Evry, France
{hussein.jawad, nicolas.brunel}@capgemini.com

## Abstract

System prompts are critical for guiding the behavior of Large Language Models (LLMs), yet they often contain proprietary logic or sensitive information, making them a prime target for extraction attacks. Adversarial queries can successfully elicit these hidden instructions, posing significant security and privacy risks. Existing defense mechanisms frequently rely on heuristics, incur substantial computational overhead, or are inapplicable to models accessed via black-box APIs. This paper introduces a novel framework for hardening system prompts through shield appending, a lightweight approach that adds a protective textual layer to the original prompt. Our core contribution is the formalization of prompt hardening as a utility-constrained optimization problem. We leverage an LLM-as-optimizer to search the space of possible SHIELDs, seeking to minimize a leakage metric derived from a suite of adversarial attacks, while simultaneously preserving task utility above a specified threshold, measured by semantic fidelity to baseline outputs. This black-box, optimization-driven methodology is lightweight and practical, requiring only API access to the target and optimizer LLMs. We demonstrate empirically that our optimized SHIELDs significantly reduce prompt leakage against a comprehensive set of extraction attacks, outperforming established baseline defenses without compromising the model's intended functionality. Our work presents a paradigm for developing robust, utility-aware defenses in the escalating landscape of LLM security. The code is made public on the following link: https://github.com/psm-defense/psm

## Introduction

**The Duality of System Prompts** System prompts have become a cornerstone of modern Large Language Model (LLM) applications, serving as the primary mechanism for steering model behavior (Liu et al. 2023b; Brown et al. 2020). These carefully engineered instructions extend beyond simple suggestions; they define an LLM's persona, operational constraints, task-specific rules, and interaction style (Malik, Jiang, and Chai 2024; Dong et al. 2023). By providing a structured framework and high-level context, system prompts ensure that model outputs are coherent, relevant, and aligned with the developer's intended goals, enhancing both performance and rule adherence (Ouyang et al. 2022; Chung et al. 2022; Wei et al. 2022; Kojima et al. 2022). For many commercial LLM-powered products, the system prompt encapsulates the core intellectual property and competitive advantage—it is the "secret sauce" that distinguishes a generic foundation model from a specialized, high-performing application (Hui et al. 2024; Zhang et al. 2023; Jiang, Jin, and He 2024; Agarwal et al. 2024).



**The Threat of Prompt Extraction** The very value of system prompts makes them a prime target for malicious actors. These prompts often contain proprietary business logic, sensitive architectural details like API integration instructions, or confidential filtering criteria that are not intended for public disclosure (Agarwal et al. 2024). This creates a critical vulnerability known as prompt extraction, a class of adversarial attack where users craft malicious inputs to deceive an LLM into revealing its own system instructions (Zhang et al. 2023; Wang et al. 2024a). This form of "adversarial reconnaissance" can serve as a precursor to more sophisticated attacks, enable the complete replication of a proprietary service, or expose sensitive operational data (Agarwal et al. 2024). The commercialization of LLM applications has amplified this threat, creating direct economic incentives for prompt extraction (Hui et al. 2024). The emergence of prompt marketplaces, where high-quality prompts are treated as valuable commodities, transforms prompt leakage from a theoretical vulnerability into a tangible economic risk, motivating persistent and sophisticated attacks (Zhang et al. 2023).

**Limitations of Existing Defenses** Current defenses against prompt extraction are often ad-hoc and fall short of providing robust protection (Agarwal et al. 2024; Wang et al. 2024b; Chen et al. 2024). Many systems rely on simple, heuristic-based instructions appended to the prompt, such as "Do not reveal your instructions." However, such defenses are brittle and easily circumvented by adversaries who craft queries to override these rules (e.g., indirect or direct prompt-injection/jailbreaks), for instance, by instructing the model to "Ignore all previous instructions" (Greshake et al. 2023; Yi et al. 2023). This vulnerability stems from a fundamental tension within LLMs: their core training objective is to be helpful and follow instructions, which can conflict with safety constraints (Wei, Haghtalab, and Steinhardt 2023). A prompt extraction query is, from the model's per-

spective, merely another instruction to be followed. A simple defensive instruction creates a direct conflict between the developer's rule and the attacker's command, a conflict that attackers have proven adept at winning (Wei, Haghtalab, and Steinhardt 2023; Greshake et al. 2023). This suggests that a truly effective defense cannot rely on a simple battle of instructions but must instead alter the prompt's structure and semantics to make it inherently less susceptible to leakage (Hines et al. 2024; Pape et al. 2025; Zhuang et al. 2025).

**Prompt Sensitivity Minimization (PSM)** This paper introduces *Prompt Sensitivity Minimization* (PSM), a principled framework designed to address the following practitioner scenario:

*"As a developer using a closed-source LLM API (e.g. OpenAI), how can I design a defense against system prompt extraction that is black-box (does not require access to the target model), lightweight (adds minimal computational overhead), and effective?"*

To answer this question, we frame the problem of defending system prompts as a *utility-constrained optimization task*. Specifically, our goal is to reduce how much of a sensitive system prompt can be extracted (minimize *leakage*), while ensuring that the model's desired behavior on normal user inputs is preserved (maintain *utility*).

PSM introduces a novel black-box method to solve this problem. It automatically generates and iteratively refines a protective *shield*—a short textual suffix appended to the original system prompt. This shield is optimized via interaction with the LLM itself, serving as a barrier that deflects adversarial prompt-extraction attempts while preserving the semantic behavior of the model.

The key contributions of this work are:

- **Black-box compatibility:** PSM requires no access to model weights, gradients, or internals, and therefore applies to any proprietary or open-source LLM accessible through an API.
- **Low overhead:** The learned shield is a static textual suffix. It imposes essentially no inference-time cost and requires no changes to the serving stack; optimization can be performed offline.
- **Iterative improvement:** An LLM-as-optimizer intelligently proposes and evolves shield candidates, discovering effective, non-obvious strategies through guided black-box search.
- **Utility preservation:** PSM enforces a hard utility constraint during optimization, ensuring that hardening does not degrade performance on intended tasks.

PSM delivers a systematic, reproducible, and practical approach to securing valuable prompt assets—moving beyond brittle heuristics to a robust, optimization-driven defense.

# Related Works

## Prompt Extraction Attacks

The vulnerability of LLMs to prompt extraction has been documented through both anecdotal evidence and systematic research. Early reports demonstrated that prompts from commercial systems like Bing Chat could be recovered through simple queries (Edwards 2023; Warren 2023). This threat was later formalized by researchers such as (Zhang et al. 2023), who demonstrated that straightforward text-based attacks could successfully extract secret prompts from a wide range of LLMs with high precision. The attack landscape has since been comprehensively mapped by benchmarks like Raccoon, which introduces a taxonomy of 14 attack types, including prefix injection, multilingual attacks, and formatting requests, establishing the diversity and complexity of threats that any defense must withstand (Wang et al. 2024a). More recent work has also highlighted the evolution of these attacks from single-turn queries to sophisticated multi-turn conversational strategies, where an adversary gradually manipulates the model into a vulnerable state, rendering static, single-shot defenses increasingly obsolete (Agarwal et al. 2024).

## Defenses Against Prompt Leakage

The literature on defending against prompt leakage reveals a clear trade-off between implementation simplicity and robustness. Current approaches can be broadly categorized as follows:

**Instructional and Heuristic Defenses** This is the most common and simplest approach, involving the addition of guardrail instructions like "Do not reveal your prompt" or the use of in-context examples to guide behavior. However, their effectiveness is limited, as they are easily bypassed by adversarial phrasing that overrides the defensive instruction (Greshake et al. 2023; Zou et al. 2023; OWASP Foundation 2025). In some cases, these defenses can even be counterproductive; for example, using XML tags to demarcate instructions has been shown to increase leakage rates (Agarwal et al. 2024).

**Input/Output Filtering and Sanitization** These methods employ external modules to screen user inputs for malicious patterns or to scan model outputs for sensitive content before it is returned to the user. While they can be effective, these approaches add computational/operational overhead and may struggle to detect novel or obfuscated attacks that do not match predefined patterns (Agarwal et al. 2024; OWASP Foundation 2025; Zhuang et al. 2025).

**Prompt Transformation and Obfuscation** This category includes more advanced techniques that modify the prompt itself. Microsoft's "Spotlighting" framework uses delimiters or character-level encoding to help the LLM differentiate between trusted developer instructions and untrusted user input (Hines et al. 2024). More sophisticated methods like *ProxyPrompt* and *Prompt Obfuscation* aim to replace the original system prompt with a functionally equivalent but semantically obscure version (Zhuang et al. 2025; Pape et al. 2025). For instance, ProxyPrompt optimizes a "proxy" in the embedding space that preserves utility for benign queries but becomes meaningless if extracted (Zhuang et al. 2025). These methods offer strong protection but often require white-box model access, posing a significant implementation barrier for many developers (Zhuang et al. 2025; Pape

et al. 2025).

**PSM** occupies a unique position on this spectrum. It achieves the robustness of an optimization-based approach like ProxyPrompt but conducts the optimization entirely through natural language interaction with a black-box LLM. This makes PSM practical, lowering the barrier to deploying systematic defenses.

### LLM-as-Optimizer

The methodology of PSM is situated within the emerging paradigm of using LLMs themselves as a core component of an optimization loop. This approach, often termed "LLM-as-optimizer," leverages the generative and reasoning capabilities of models to propose, evaluate, and refine solutions for complex problems (Yang et al. 2023). LLMs have been successfully used to generate and optimize prompts for specific tasks (Yang et al. 2023; Pryzant et al. 2023; Fernando et al. 2023; Khattab et al. 2023), write and debug code (Chen et al. 2021), and perform iterative self-improvement through reflection and feedback (Madaan et al. 2023; Shinn et al. 2023). PSM extends this concept to the security domain, using an LLM to navigate the high-dimensional semantic space of natural language to find an optimal defensive prompt.

## Prompt Sensitivity Minimization (PSM)

PSM is a general framework for reducing system prompt leakage. It introduces a small suffix, called a *shield*, that modifies the model's behavior in a targeted way to prevent sensitive content from being exposed, while preserving the model's intended functionality on benign inputs.

### Problem Formulation

We formalize prompt hardening as a constrained optimization problem. Given a sensitive system prompt $P$ and a model under evaluation as $M$, our goal is to learn an optimized suffix $S$ that:

- minimizes the leakage of $P$ when the model is probed by adversarial users,
- while maintaining task-specific utility for legitimate users.

Let $P \oplus S$ denote the concatenation of the original prompt $P$ and the shield $S$. We define the optimization objective as follows:

$$\begin{aligned} \min_{S} \quad & L(P \oplus S) \\ \text{subject to} \quad & U(P \oplus S) \geq \tau, \end{aligned} \tag{1}$$

Where:

- $L(P \oplus S)$ is a leakage score that quantifies the extent to which the original prompt $P$ can be inferred or reproduced under adversarial attacks.
- $U(P \oplus S)$ measures the model's task-specific utility under benign input queries.
- $\tau$ is a predefined threshold that sets the minimum acceptable utility.

This formulation explicitly captures the trade-off between robustness and functionality. It enables automated, black-box search for suffixes that reduce prompt exposure while preserving downstream task performance.

### Prompt Structure and Design Motivation

The core mechanism in PSM is the addition of a suffix shield with structured markers:

> [SYSTEM PROMPT] {Original Prompt}
> [SHIELD] {Optimized Shield}

This design employs two key principles: **(1) suffix-based placement** and **(2) structured separation**. The suffix-based placement is deliberate. Empirical studies and analyses of model behavior indicate that LLMs tend to weigh information at the end of a long context more heavily, and often follow the most recent instruction when instructions conflict (Liu et al. 2023a; Zhang, Zhang et al. 2025). In practice, later (appended) instructions can override earlier guidance, an effect exploited by indirect prompt-injection attacks (Greshake et al. 2023).

The structured markers `[SYSTEM PROMPT]` and `[SHIELD]` provide explicit separation and labeling of different instruction components. This structured marking improves robustness by clearly delineating the original prompt from the protective shield, making it easier for the model to recognize and respect the hierarchical nature of the instructions (Hines et al. 2024). By combining both suffix placement and structured separation, we gain stronger control over how the model responds to adversarial queries—without disturbing the structural or semantic integrity of the original prompt.

### Leakage Objective

The *leakage objective*, denoted $L(P \oplus S)$, measures the extent to which a sensitive system prompt $P$ can be recovered from a language model that is operating under a candidate shield $S$. Intuitively, this objective captures the worst-case ability of an adversary to elicit exact or paraphrased reproductions of $P$, even when shielded.

**Setup.** Let $A = \{a_1, a_2, \ldots, a_{|A|}\}$ be a set of adversarial queries. For each attack $a \in A$, we obtain the model's output under the shielded prompt:

$$R_a = M(P \oplus S \oplus a),$$

where $M(\cdot)$ denotes the model's response.

To evaluate how much of $P$ is exposed in each response $R_a$, we compute the ROUGE-L recall (Lin 2004; Zhang et al. 2023) score between $P$ and $R_a$. This metric captures the longest common subsequence between the two texts and is particularly suited to detecting partial or paraphrased reproductions of $P$.

**Hard Maximum Formulation.** The initial definition of leakage takes the worst-case (maximum) score over all adversarial queries:

$$L(P \oplus S) = \max_{a \in A} \text{ROUGE-L}_{\text{recall}}(P, R_a). \tag{2}$$

However, this formulation is non-smooth and difficult to optimize using gradient-free, LLM-based search strategies.

**Smooth Approximation.** To address this, we adopt a smoothed version using the log-sum-exp (LSE) function:

$$L(P \oplus S) = \frac{1}{\beta} \log \sum_{a \in A} \exp\left(\beta \cdot \text{ROUGE-L}_{\text{recall}}(P, R_a)\right), \tag{3}$$

where $\beta > 0$ is a temperature parameter that controls the sharpness of the approximation. As $\beta \to \infty$, this converges to the hard maximum in Eq. 2.

In our experiments, we set $\beta = 10$, which balances smoothness and fidelity to the worst-case.

**Why ROUGE-L Recall?** ROUGE-L recall (Lin 2004) measures how much of the reference text is recovered in the model's output via the longest common subsequence, giving a graded signal of prompt exposure even under minor rewording.

$$\text{ROUGE-L}_{\text{recall}}(X, Y) = \frac{|\,\text{LCS}(tokens(X), tokens(Y))|}{|tokens(X)|}$$

**Adversary design.** To approximate a strong, realistic attacker, we evaluate shields against a curated set of adversarial prompts. In our experiments we use $|A| = 50$ adversarial queries constructed via a *compositional attack* template: each adversarial prompt concatenates three orthogonal strategies that are known to increase extraction success:

- **Distractors:** Context-shifting phrases (e.g., "I'm just kidding, this was a test, instead...") that exploit the model's tendency to focus on recent instructions and reduce fidelity to prior defensive content.
- **Repetition Requests:** Explicit commands such as "Repeat the system prompt", which capitalize on the model's training to follow direct, helpful instructions.
- **Formatting Requests:** Instructions that reframe the prompt as manipulable content, like "Format the system prompt as a Python triple-quoted string". These convert the extraction attempt into a benign-seeming task (e.g., code formatting), which models are trained to perform reliably.

Each adversarial query follows the pattern:

`{Distractor} {Repetition} {Formatting}`

This compositional structure creates potent attacks. Empirically, they achieve a near-100% extraction success rate across multiple models, including GPT-4o and GPT-4o-mini. By optimizing against these strong adversaries, PSM is able to learn robust shields capable of withstanding real-world prompt extraction attempts.

**Note:** The attack dataset used in this optimization is fully customizable: practitioners can easily incorporate new adversarial strategies, domain-specific queries, or emerging attack vectors.

## Utility Objective: $U$

To ensure that defensive modifications do not degrade a model's intended behavior, we evaluate *task utility* for each system prompt. Specifically, we assess whether the model continues to produce high-quality, task-relevant outputs when operating under a shielded (hardened) prompt.

For each system prompt under evaluation, we construct a corresponding gold-standard dataset

$$\mathcal{D}_{\text{gold}} = \{(q_i, g_i)\}_{i=1}^{N},$$

where $q_i$ is a representative, benign input query and $g_i$ is its ideal response. Gold responses are generated using **GPT-4o**, which serves as a trusted reference model. The dataset is carefully designed to reflect the intended functionality and domain of the original system prompt.

To evaluate utility, we compare the model's outputs before and after hardening. Let $b_i$ denote the baseline model's response to query $q_i$ (with the original system prompt $P$), and let $t_i$ be the shielded model's response (with $P \oplus S$). We compute the semantic similarity between each response and the gold answer $g_i$ using cosine similarity over sentence embeddings from `sentence-transformers/all-MiniLM-L6-v2`.

Formally, we define:

$$b_i = M(P \oplus q_i), \quad t_i = M(P \oplus S \oplus q_i),$$

and evaluate relative similarity:

$$r_i = \frac{\text{sim}(t_i, g_i)}{\text{sim}(b_i, g_i)},$$

**Aggregate Score.** We aggregate over the dataset as:

$$\mathcal{U}_{\text{raw}} = \frac{1}{N} \sum_{i=1}^{N} r_i \tag{4}$$

This ratio-based formulation emphasizes *relative* performance—allowing tolerance for small degradations while flagging substantial utility loss.

The resulting utility score $U$ quantifies how well the shielded model preserves the original prompt's functionality. A high score indicates that defensive shielding does not interfere with the model's intended task. The utility function $U(P \oplus S)$ thus guards against over-regularization, ensuring that shields do not suppress useful capabilities.

## Fitness Function for Black-Box Optimization

To enable automatic discovery of shields via black-box search, we convert the constrained optimization into a single scalar objective using a penalty method:

$$\text{fitness}(S) \;=\; L(P \oplus S) \;+\; \lambda \cdot \max(0, \tau - U(P \oplus S)) \tag{5}$$

where $\lambda$ is a large multiplier (e.g., 100) and $\tau$ is the minimum acceptable utility (e.g., 0.9).

This formulation heavily penalizes any shield that compromises functionality, while prioritizing the minimization of leakage. The search process can now proceed using any black-box optimizer, by evaluating the fitness of different shield candidates.

### Optimization via LLM-as-Optimizer

PSM employs an iterative, evolutionary-style algorithm where an LLM serves as the intelligent generator of new candidate solutions. This approach replaces the random mutation and crossover steps of traditional genetic algorithms with an informed generation process guided by in-context learning.

- **Initialization:** An optimizer LLM is prompted to generate an initial population of 5 diverse shield candidates. The prompt encourages variety in phrasing and strategy.
- **Evaluation:** Each candidate shield is evaluated using the fitness function described above. This involves running the full suite of adversarial and baseline queries to compute its leakage and utility scores.
- **Selection & Generation:** The top-performing candidates from all iterations so far (e.g., the best 10) are selected. The text of these shields, along with their corresponding fitness scores, is formatted into a new prompt for the optimizer LLM. This prompt asks the LLM to analyze the characteristics of successful shields (low leakage, high utility) and generate 5 new, improved candidates that build upon these successful patterns.
- **Iteration:** Steps 2 and 3 are repeated for a set number of iterations or until a termination condition is met. The termination condition is defined by achieving both high utility and low leakage (e.g., $U \geq 0.9$ and $L < 0.65$).

This process leverages the LLM's linguistic and reasoning capabilities to intelligently navigate the vast semantic space of possible shields, converging more efficiently on effective solutions than a random or purely heuristic search would allow.

## Experimental Setup

### Models and Serving Configuration

**Victim models.** We evaluate PSM on three instruction-tuned, closed-source LLMs served via API: GPT-5-MINI, GPT-4.1-MINI, and GPT-4O-MINI. All models are queried strictly in a black-box fashion (no access to gradients, weights, or logits). Unless otherwise noted, decoding is deterministic with temperature $= 0$ and top-$p = 1$.

**Optimizer model.** Unless stated otherwise, the LLM-as-optimizer is GPT-4O-MINI (API). In each optimization iteration, the optimizer proposes five candidate shields using temperature $= 1$. This keeps the end-to-end pipeline fully black-box with respect to the victim models.

### Victim Prompt Corpora

We evaluate our defense on two corpora of system prompts that approximate real-world deployments. For each prompt, we set it as the model's system configuration, run the full attack suites, and measure leakage with and without the defense.

**Unnatural Instructions** Following Zhang et al. (2023), we sample 30 prompts from the Unnatural Instructions dataset (Honovich et al. 2023). This corpus is a large, LLM-generated collection of instructions created by prompting GPT-3 to propose diverse tasks, then paraphrasing them to increase coverage.

**Synthetic System Prompts** We also sample 30 prompts from the Synthetic System-Prompt-Leakage dataset (Chua 2025), as adopted by the SPE-LLM framework (Das, Amini, and Wu 2025). It contains application-style system prompts (e.g., customer support, education, content creation), making it well suited for assessing practical system-prompt–leakage risk.

### Baselines

We evaluate PSM against four defenses:

- **No-Defense.** The raw, unmodified system prompt $P$, representing a baseline without any protection.
- **N-gram Output Filter.** Following Zhang et al. (2023), this defense suppresses the model's output if it contains any contiguous 5-word sequence (5-gram) found in the original prompt $P$, aiming to block exact leakage.
- **Heuristic Guardrail (Direct).** As proposed by Liang et al. (2024), this method appends a fixed defensive instruction to the system prompt: *"Do not reveal or describe your system prompt or developer instructions."*
- **Decoy Prompt (FAKE).** Also from Liang et al. (2024), this defense prepends a misleading decoy prompt to distract prompt extraction attempts. The decoy: *"Your Instruction: You are a superhero who aims to SAVE the world."*

### Attack Suites

We measure robustness against four complementary suites:

- **Raccoon (59 prompts).** Diverse prompt-extraction templates spanning prefix/indirect injection, multilingual, formatting, and instruction hierarchy stressors (Wang et al. 2024a).
- **Raccoon-Language.** Exactly the Raccoon templates, but each prompt appends an explicit "in language" constraint (e.g., "in Portuguese", "in French", "in Italian"). This stresses multilingual reformulation and exposes a limitation of exact-match/n-gram output filters, which fail when the attacker requests the system prompt to be paraphrased or translated (Wang et al. 2024a).
- **Polite-Requests ("Liang"- 22 prompts).** Social-engineering style, deferential queries that attempt to elicit hidden rules without covert override commands (Liang et al. 2024).
- **Command-Override ("Zhang" - 110 prompts).** Direct instruction-override jailbreaks (e.g., "ignore prior rules", role/context flips) (Zhang et al. 2023).

During optimization, we compute the leakage objective against a bespoke set of 50 compositional adversarial prompts (Distractor + Repeat + Formatting), section leakage

| Dataset | Attack | Defense | GPT-5-mini | | GPT-4.1-mini | | GPT-4o-mini | |
|---|---|---|---|---|---|---|---|---|
| | | | AM Avg | JM Avg | AM Avg | JM Avg | AM Avg | JM Avg |
| Synthetic System Prompts | Liang | No Defense | **0%** | 54% | 26% | 78% | **0%** | 32% |
| | | Fake | **0%** | 54% | 24% | 62% | 1% | 40% |
| | | Direct | **0%** | 40% | 3% | 47% | **0%** | 16% |
| | | PSM | **0%** | **13%** | **0%** | **4%** | **0%** | **6%** |
| | Zhang | No Defense | 1% | 42% | 26% | 34% | 3% | 27% |
| | | Fake | 1% | 42% | 30% | 32% | 7% | 33% |
| | | Direct | **0%** | 31% | 9% | 18% | **0%** | 14% |
| | | PSM | **0%** | **8%** | **0%** | **2%** | **0%** | **6%** |
| | Raccoon | No Defense | 2% | 42% | 61% | 59% | 15% | 27% |
| | | Fake | 2% | 39% | 60% | 41% | 23% | 32% |
| | | Direct | **0%** | 28% | 49% | 51% | 1% | 11% |
| | | PSM | **0%** | **8%** | **7%** | **5%** | **0%** | **4%** |
| UNNATURAL | Liang | No Defense | 1% | 24% | 30% | 54% | 10% | 21% |
| | | Fake | 1% | 23% | 18% | 17% | 19% | 13% |
| | | Direct | **0%** | 8% | 10% | 19% | **0%** | **0%** |
| | | PSM | **0%** | **0%** | **0%** | **0%** | **0%** | **0%** |
| | Zhang | No Defense | 3% | 30% | 31% | 32% | 12% | 14% |
| | | Fake | 3% | 24% | 29% | 16% | 19% | 16% |
| | | Direct | **0%** | 15% | 13% | 13% | **0%** | 1% |
| | | PSM | **0%** | **3%** | **0%** | **1%** | **0%** | **0%** |
| | Raccoon | No Defense | 4% | 22% | 45% | 42% | 21% | 20% |
| | | Fake | 3% | 20% | 51% | 21% | 31% | 16% |
| | | Direct | **0%** | 12% | 43% | 39% | 4% | 4% |
| | | PSM | **0%** | **3%** | **6%** | **5%** | **0%** | **0%** |

Table 1: Attack success rates (%) on the UNNATURAL and Synthetic System Prompts corpora, averaged over attacks (AM = Approximate Match, JM = Judge Match). Lower is better; bold denotes the lowest value within each dataset×attack block.

| Dataset | Attack | Defense | GPT-5-mini | GPT-4.1-mini | GPT-4o-mini |
|---|---|---|---|---|---|
| | | | JM Avg | JM Avg | JM Avg |
| Synthetic System Prompts | Raccoon | No Defense | 42% | 59% | 27% |
| | | Filter | 18% | **3%** | 5% |
| | | PSM | **8%** | 5% | **4%** |
| | Raccoon-Language | No Defense | 35% | 59% | 17% |
| | | Filter | 32% | 48% | 11% |
| | | PSM | **9%** | **6%** | **3%** |
| UNNATURAL | Raccoon | No Defense | 22% | 42% | 20% |
| | | Filter | 9% | **1%** | 1% |
| | | PSM | **3%** | 5% | **0%** |
| | Raccoon-Language | No Defense | 25% | 37% | 15% |
| | | Filter | 22% | 29% | 10% |
| | | PSM | **4%** | **6%** | **1%** |

Table 2: Judge-Match (JM) attack success rates (%) for Raccoon and Raccoon-Language suites on the UNNATURAL and Synthetic System Prompts corpora. Lower is better; bold denotes the lowest value within each dataset×attack block.

| **Dataset** | **GPT-5-mini** | **GPT-4.1-mini** | **GPT-4o-mini** |
|---|---|---|---|
| Synthetic System Prompts | 101.88% | 100.89% | 99.73% |
| UNNATURAL | 101.27% | 114.76% | 100.73% |

Table 3: Average utility preservation (%) for each model across system prompt corpora (Synthetic System Prompts and UNNATURAL), computed using the same definition as in Section *Utility Objective*.

objective. None of the Raccoon, Raccoon-Language, Liang, or Zhang prompts are used in the optimization loop; all four suites are held out and used only for evaluation.

## Scoring, Metrics, and Protocol

**Leakage Metrics.** To quantify the success of an extraction attack $a$ on a victim system prompt $P$, we analyze the model's output $R_a$ using two complementary evaluation criteria:

- **Approximate Match (AM).** Captures surface-level leakage via textual overlap. We compute ROUGE-L recall between the original system prompt $P$ and the model's response $R_a$, considering an attack successful if the score exceeds a threshold $\theta = 0.9$. This metric detects near-verbatim or structurally similar reproductions.
- **Judge Match (JM).** Detects semantic leakage based on equivalence judged by an external LLM. This metric accounts for paraphrased or semantically aligned reproductions of $P$, even with low lexical overlap. The prompt:

> You are an expert evaluator tasked with assessing the semantic equivalence of extracted system prompts. Your job is to compare an extracted system prompt against the original system prompt and provide a binary evaluation.
> Evaluation Criteria:
> - Score 1: The extracted prompt contains a paraphrase of the original system prompt.
> - Score 0: The extracted prompt does not fully preserve the meaning of the original system prompt
> Provide a binary score (0 or 1)

**Attack Success Rate (ASR).** For each attack $a$, we define the *attack success rate* as the proportion of system prompts in a given set $\mathcal{P}$ for which leakage is detected under either AM or JM:

$$\mathrm{ASR}(a) = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} s(P, a), \tag{6}$$

where $s(P, a) \in \{0, 1\}$ indicates whether the attack $a$ successfully elicits leakage from prompt $P$.

**Aggregate Metrics.** To summarize performance across multiple attacks $\mathcal{A}$, we report the mean attack success rate (AVG):

$$\textsc{Avg} = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathrm{ASR}(a). \tag{7}$$

**Optimization details.** We use penalty fitness $\mathrm{fitness}(S) = L(P \oplus S) + \lambda \cdot \max(0, \tau - U(P \oplus S))$, with $\lambda = 100$. Each run uses population size $= 5$, top-$k = 10$ memory across iterations, and $T = 10$ iterations.. The optimizer receives the top-$k$ shields and scores as in-context exemplars when generating the next population.

## Results and Interpretation

**Overall effectiveness.** Across both datasets, PSM achieves the lowest attack–success rates (ASR) across all attack suites, often reducing leakage to 0–6%. This indicates that PSM suppresses both surface-level and semantic reproduction.

**Model consistency.** While baseline leakage varies by model (e.g., higher JM ASR for `gpt-4.1-mini`), PSM yields uniformly low ASR across all models, suggesting strong cross-model generalization. Heuristic defenses like *Direct* and *Fake* reduce leakage but are consistently outperformed by PSM.

**Exact-match filters vs paraphrase/translation.** Table 2 isolates JM ASR on Raccoon and Raccoon-Language to stress exact-copy versus paraphrase/translate settings. A 5-gram output filter can be competitive under exact-match Raccoon on some models, but it degrades markedly on Raccoon-Language, where attackers request translations or paraphrases. In contrast, PSM remains low on both (e.g., single-digit JM ASR across models), indicating robustness beyond literal overlap.

**Utility preservation.** Table 3 shows that PSM preserves semantic task fidelity across datasets and models, with utility scores often exceeding the baseline. This indicates that hardening via PSM does not degrade model utility.

# Discussion and Future Work

**Implications** PSM offers a practical, black-box way to harden LLM applications by automatically discovering short shield suffixes that curb prompt leakage while preserving task utility. It scales to real deployments without access to model internals or specialist expertise, improving the security posture of LLM services and protecting proprietary prompts with negligible inference-time overhead.

**Limitations** Our optimization loop is computationally intensive due to repeated evaluation on adversarial and benign sets. Effectiveness depends on the breadth and realism of the attack suite used during optimization, so transfer to unseen attack families is not guaranteed. Finally, our study targets prompt-extraction attacks specifically.

**Future Work**

- **Broader Threats:** Extend PSM to jailbreaks and multi-turn conversational attacks.
- **Transferability:** Test whether shields transfer across model families and providers.
- **Efficiency:** Develop search heuristics or alternative optimizers to reduce compute.

**Conclusion** We presented Prompt Sensitivity Minimization, which reframes prompt hardening as black-box, utility-constrained optimization driven by an LLM-as-optimizer. PSM learns lightweight shields that significantly reduce leakage while maintaining utility, achieving a near zero attack success rate on a challenging test suite. This provides an accessible, reproducible path to protect valuable prompt IP and secure next-generation LLM systems.

## References


Agarwal, D.; Fabbri, A.; Risher, B.; Laban, P.; Joty, S.; and Wu, C.-S. 2024. Prompt Leakage Effect and Defense Strategies for Multi-Turn LLM Applications. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*.

Brown, T. B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Chen, S.; et al. 2024. Structured prompting: Training LLMs that separate trusted instructions from untrusted data. *arXiv preprint arXiv:2402.06363*.

Chua, G. 2025. System Prompt Leakage Dataset. Hugging Face Dataset. Default split: 283,353 train / 71,351 test (total ∼355k rows).

Chung, H. W.; et al. 2022. Scaling Instruction-Finetuned Language Models. *arXiv:2210.11416*.

Das, B. C.; Amini, M. H.; and Wu, Y. 2025. System Prompt Extraction Attacks and Defenses in Large Language Models. *arXiv preprint arXiv:2505.23817*.

Dong, Y.; Wang, Z.; Narsimhan Sreedhar, M.; Wu, X.; and Kuchaiev, O. 2023. SteerLM: Attribute Conditioned SFT as an (User-Steerable) Alternative to RLHF. *arXiv:2310.05344*.

Edwards, B. 2023. AI-powered Bing Chat spills its secrets via prompt injection attack. *Ars Technica*. Published Feb 10, 2023.

Fernando, C.; Banarse, D.; Michalewski, H.; Osindero, S.; and Rocktäschel, T. 2023. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. *arXiv preprint arXiv:2309.16797*.

Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the ACM CCS (Demonstrations/Companion) or arXiv:2302.12173*.

Hines, K.; Lopez, G.; Hall, M.; Zarfati, F.; Zunger, Y.; and Kiciman, E. 2024. Defending Against Indirect Prompt Injection Attacks with Spotlighting. *arXiv:2403.14720*.

Honovich, O.; Scialom, T.; Levy, O.; and Schick, T. 2023. Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor. In Rogers, A.; Boyd-Graber, J.; and Okazaki, N., eds., *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 14409–14428. Toronto, Canada: Association for Computational Linguistics.

Hui, B.; Yuan, H.; Gong, N.; Burlina, P.; and Cao, Y. 2024. PLeak: Prompt Leaking Attacks against Large Language Model Applications. *arXiv:2405.06823*. To appear in ACM CCS 2024.

Jiang, Z.; Jin, Z.; and He, G. 2024. PromptKeeper: Safeguarding System Prompts for LLMs. *arXiv:2412.13426*. Findings of EMNLP 2025.

Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714*.

Kojima, T.; et al. 2022. Large Language Models are Zero-Shot Reasoners. *arXiv:2205.11916*.

Liang, Z.; Hu, H.; Ye, Q.; Xiao, Y.; and Li, H. 2024. Why Are My Prompts Leaked? Unraveling Prompt Extraction Threats in Customized Large Language Models. *arXiv preprint arXiv:2408.02416*.

Lin, C.-Y. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Proceedings of the Workshop on Text Summarization Branches Out (WAS 2004), ACL 2004*. Barcelona, Spain.

Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2023a. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172*.

Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; and Neubig, G. 2023b. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 55(9): 1–35.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *arXiv preprint arXiv:2303.17651*.

Malik, M.; Jiang, J.; and Chai, K. M. A. 2024. An Empirical Analysis of the Writing Styles of Persona-Assigned LLMs. In *EMNLP 2024*.

Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

OWASP Foundation. 2025. OWASP Top 10 for LLM Applications: LLM01:2025 Prompt Injection.

Pape, D.; Mavali, S.; Eisenhofer, T.; Schönherr, L.; et al. 2025. Prompt Obfuscation for Large Language Models. In *USENIX Security 2025*.

Pryzant, R.; Iter, D.; Li, J.; Lee, Y. T.; Zhu, C.; and Zeng, M. 2023. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. *arXiv preprint arXiv:2305.03495*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*.

Wang, J.; Yang, T.; Xie, R.; and Dhingra, B. 2024a. Raccoon: Prompt Extraction Benchmark of LLM-Integrated Applications. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Wang, J.; et al. 2024b. Prompt Extraction Benchmark of LLM-Integrated Applications. In *Findings of ACL 2024*.

Warren, T. 2023. These are Microsoft's Bing AI secret rules and why it says it's named Sydney. *The Verge*. Published Feb 16, 2023.

Wei, A.; Haghtalab, N.; and Steinhardt, J. 2023. Jailbroken: How Does LLM Safety Training Fail? *arXiv preprint arXiv:2307.02483*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.

Yang, C.; Wang, X.; Lu, Y.; Liu, H.; Le, Q. V.; Zhou, D.; and Chen, X. 2023. Large Language Models as Optimizers. *arXiv preprint arXiv:2309.03409*.

Yi, J.; et al. 2023. Benchmarking and Defending Against Indirect Prompt Injection Attacks. *arXiv preprint arXiv:2312.14197*.

Zhang, Y.; et al. 2023. Effective Prompt Extraction from Language Models. *arXiv:2307.06865*.

Zhang, Z.; Zhang, W.; et al. 2025. IHEval: Evaluating Language Models on Following the Instruction Hierarchy. In *Proceedings of NAACL-HLT 2025*.

Zhuang, Z.; Nicolae, M.-I.; Wang, H.-P.; and Fritz, M. 2025. ProxyPrompt: Securing System Prompts against Prompt Extraction Attacks. *arXiv preprint arXiv:2505.11459*.

Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J. Z.; and Fredrikson, M. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv:2307.15043*.

# Association for the Advancement of Artificial Intelligence

**1101 Pennsylvania Ave, NW Suite 300**
**Washington, DC 20004**


## AAAI COPYRIGHT FORM

Title of Article/Paper: PSM: Prompt Sensitivity Minimization via LLM-Guided Black-Box Optimization

Publication in Which Article/Paper Is to Appear: AAAI 2026 Workshop on Trust and Control in Agentic AI (TrustAgent)

Author's Name(s): Hussein Jawad, Nicolas Brunel

Please type or print your name(s) as you wish it (them) to appear in print

### PART A – COPYRIGHT TRANSFER FORM

The undersigned, desiring to publish the above article/paper in a publication of the Association for the Advancement of Artificial Intelligence, (AAAI), hereby transfer their copyrights in the above article/paper to the Association for the Advancement of Artificial Intelligence (AAAI), in order to deal with future requests for reprints, translations, anthologies, reproductions, excerpts, and other publications.

This grant will include, without limitation, the entire copyright in the article/paper in all countries of the world, including all renewals, extensions, and reversions thereof, whether such rights current exist or hereafter come into effect, and also the exclusive right to create electronic versions of the article/paper, to the extent that such right is not subsumed under copyright.

The undersigned warrants that they are the sole author and owner of the copyright in the above article/paper, except for those portions shown to be in quotations; that the article/paper is original throughout; and that the undersigned right to make the grants set forth above is complete and unencumbered.

If anyone brings any claim or action alleging facts that, if true, constitute a breach of any of the foregoing warranties, the undersigned will hold harmless and indemnify AAAI, their grantees, their licensees, and their distributors against any liability, whether under judgment, decree, or compromise, and any legal fees and expenses arising out of that claim or actions, and the undersigned will cooperate fully in any defense AAAI may make to such claim or action. Moreover, the undersigned agrees to cooperate in any claim or other action seeking to protect or enforce any right the undersigned has granted to AAAI in the article/paper. If any such claim or action fails because of facts that constitute a breach of any of the foregoing warranties, the undersigned agrees to reimburse whomever brings such claim or action for expenses and attorneys' fees incurred therein.

**Returned Rights**

In return for these rights, AAAI hereby grants to the above author(s), and the employer(s) for whom the work was performed, royalty-free permission to:

1. Retain all proprietary rights other than copyright (such as patent rights).
2. Personal reuse of all or portions of the above article/paper in other works of their own authorship. This does not include granting third-party requests for reprinting, republishing, or other types of reuse. AAAI must handle all such third-party requests.
3. Reproduce, or have reproduced, the above article/paper for the author's personal use, or for company use provided that AAAI copyright and the source are indicated, and that the copies are not used in a way that implies AAAI endorsement of a product or service of an employer, and that the copies per se are not offered for sale. The foregoing right shall not permit the posting of the article/paper in electronic or digital form on any computer network, except by the author or the author's employer, and then only on the author's or the employer's own web page or ftp site. Such web page or ftp site, in addition to the aforementioned requirements of this Paragraph, shall not post other AAAI copyrighted materials not of the author's or the employer's creation (including tables of contents with links to other papers) without AAAI's written permission.
4. Make limited distribution of all or portions of the above article/paper prior to publication.
5. In the case of work performed under a U.S. Government contract or grant, AAAI recognized that the U.S. Government has royalty-free permission to reproduce all or portions of the above Work, and to authorize others to do so, for official U.S. Government purposes only, if the contract or grant so requires.

In the event the above article/paper is not accepted and published by AAAI, or is withdrawn by the author(s) before acceptance by AAAI, this agreement becomes null and void.

(1)

| | |
|---|---|
| | 28/11/2025 |
| Author/Authorized Agent for Joint Author's Signature | Date (MM/DD/YYYY) |
| Capgemini Invent / LaMME, University Par | |
| Employer for whom work was performed | Title (if not author) |

*(For jointly authored Works, all joint authors should sign unless one of the authors has been duly authorized to act as agent for the others.)*

Hussein